UNIDAD 1

Introducción y conceptos básicos

INTRODUCCIÓN Y CONCEPTOS BÁSICOS

1.1 Programas y datos

1.2 Persistencia de datos

1.3 Sistemas de persistencia de datos

1.4 Almacenamiento de datos

1.5 Restricciones de integridad

1.6 Control de accesos concurrentes y transacciones

1.7 Persistencia de datos en ficheros

1.8 Persistencia de datos en bases de datos relacionales

1.9 Persistencia de datos en bases de datos NoSQL

1 Programas y datos







Los ordenadores ejecutan programas que gestionan información.

Esta **información se representa en forma de datos**, que pueden estar estructurados de distintas formas.

Los tipos de datos que manejan los sistemas actuales y los tipos de operaciones que realizan con ellos son muy diversas, pero siempre cuentan dos tipos de medios de almacenamiento de datos:

- **Almacenamiento primario o memoria principal**: *donde se almacenan los datos con los que está trabajando el programa en cada momento, se borran al apagarse, capacidad relativamente baja y tiempo de acceso corto*.
- **Almacenamiento secundario**: *donde se almacenan los datos de forma permanente. Los datos **almacenados en estos medios son datos persistentes**, no se borran cuando se apaga el sistema. Tienen capacidad de almacenamiento alta y tiempo de acceso a los datos relativamente largo.*

2 Persistencia de datos

Es el conjunto de técnicas, mecanismos y tecnologías que permiten **guardar y conservar la información de una aplicación de forma permanente** en un soporte físico (disco duro, base de datos, memoria no volátil, etc.), de modo que los datos no se pierdan cuando el programa se cierra, se reinicia o el sistema se apaga.

Los programas **solo pueden consultar directamente datos almacenados en memoria principal:**

- Pero pueden crear datos en memoria principal y guardarlos en almacenamientos secundarios, convirtiendo estos datos en datos persistentes.
- A la inversa también, un programa puede **recuperar datos persistentes** de un medio de almacenamiento secundario para pasarlos a la memoria principal, una vez ahí se convierten en datos transitorios.

Es muy habitual que el **medio de almacenamiento secundario esté en un ordenador distinto**, por ejemplo, un servidor que proporciona servicios de persistencia de datos a múltiples aplicaciones que se ejecutan en distintos ordenadores.

2 Persistencia de datos

Los programas de aplicación suelen acceder a los servicios para almacenamiento y consulta de datos persistentes mediante funciones de alto nivel proporcionadas por diversas APIs (interfaces de programación de aplicaciones)



3 Sistema de Persistencia de datos

Los datos persistentes se **guardan siempre en ficheros dentro de un sistema de ficheros en un medio de almacenamiento secundario.**

¿Qué es un sistema de ficheros? Una jerarquía de directorios/carpetas

Sobre estos sistemas básicos de almacenamiento se pueden organizar sistemas de datos más o menos sofisticados

Cada sistema puede ser más o menos apropiado y tendrá sus ventajas e inconvenientes para cada aplicación.

Hay que tenerlas en cuenta y evaluarlas críticamente para decidir la mejor solución.

No solo hay que tener en cuenta las necesidades actuales, sino también las futuras, escalabilidad.

4 Almacenamiento de datos

Cada sistema proporciona una estructura básica de almacenamiento para los datos.

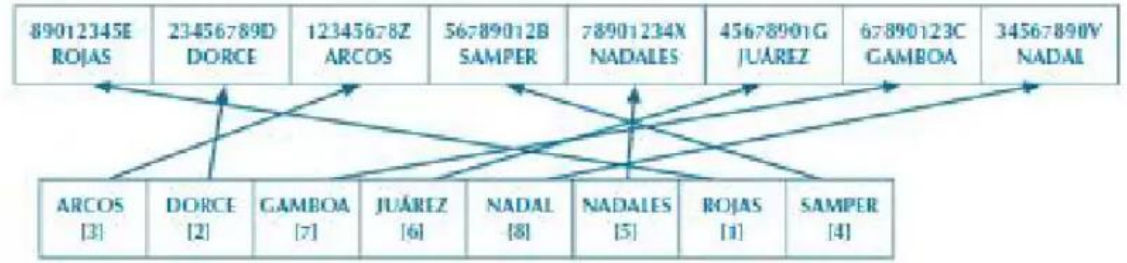
Ficheros

Los ficheros proporcionan por sí mismos una **organización secuencial de los datos**. Un fichero no es otra cosa que una secuencia de bytes.

Sobre este soporte elemental se puede representar cualquier tipo de información. De hecho, todos los sistemas de almacenamiento de datos almacenan los datos en ficheros.

Pero cuando se habla de sistemas basados en ficheros, se suele hacer referencia a los **ficheros secuenciales**, en los que la información dentro de cada fichero se almacena en una secuencia de registros de longitud fija, y cada registro está compuesto por varios campos de longitud fija.

Un ejemplo, un registro podría tener los datos de un cliente, almacenados en campos tales como DNI, nombre, dirección, etc.



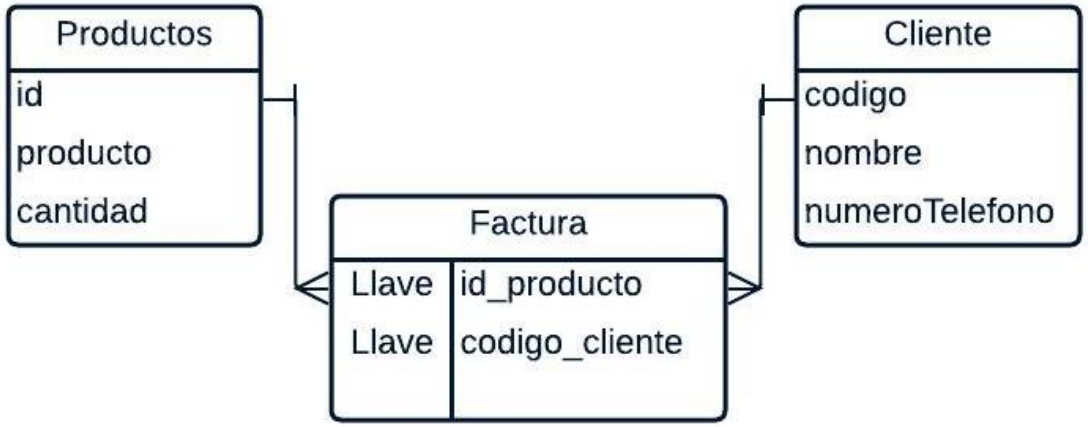
Almacenamiento de datos

Bases de datos relacionales

Estas bases de datos organizan los datos en tablas y permiten especificar las relaciones entre dichas tablas.

Hoy en día predominan las bases de datos relacionales, que en los años ochenta desplazaron a los antiguos sistemas basados en ficheros.

SQL es el lenguaje estándar para bases de datos relacionales. Por ello, frecuentemente, se denomina base de datos NoSQL a cualquier base de datos no relacional.

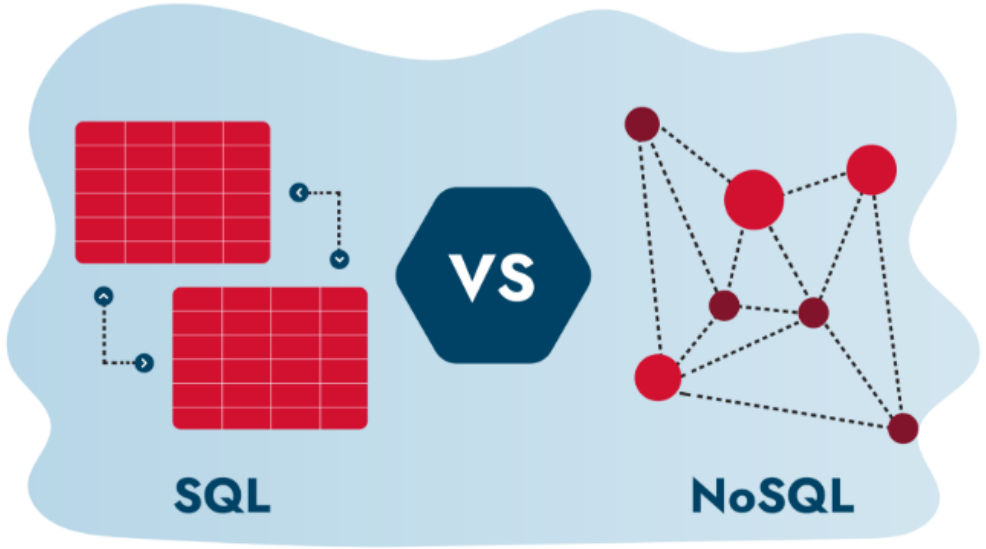


Almacenamiento de datos

Bases de datos NoSQL

Las diversas bases de datos NoSQL representan la información de manera muy diversa y, en general, diferente de los modelos comentados hasta ahora.

En general utilizan estructuras muy sencillas y a la vez muy flexibles.



Bases de datos relacionales

Ventajas:

1

2

3

Inconvenientes

1

2

3

Bases de datos NoSQL

Ventajas:

1

2

3

Inconvenientes

1

2

3

5 Restricciones de integridad

Es una **regla condicional que se define en la estructura de una base de datos** para garantizar que la información almacenada sea **precisa, válida, consistente y confiable** en todo momento.

Si un usuario o una aplicación intenta insertar, modificar o eliminar datos de una manera que **violaría estas reglas**, el **motor de la base de datos rechaza** la operación automáticamente.

Se pueden definir:

- para datos **particulares** (por ejemplo, no se permite que el nombre de un cliente no esté indicado),

- para datos **relacionados** (por ejemplo, no se permite que el cliente para una factura no exista), y eso significa que esté indicado en la factura, y además que exista como entidad independiente).

5 Restricciones de integridad

¿Por qué son fundamentales?

Prevención de datos corruptos: Evita entradas inválidas (por ejemplo, guardar texto en un campo de fecha o correos sin el formato @).

Seguridad a nivel de motor: Las reglas se aplican directamente en el Gestor de Base de Datos (DBMS), garantizando que ningún programa externo pueda romper la consistencia de los datos.

Mantenibilidad: Simplifica el código de las aplicaciones cliente, ya que la base de datos se encarga de validar las condiciones lógicas base.

6 Control de accesos concurrentes y transacciones

El **control de accesos concurrentes** es el conjunto de técnicas y mecanismos que utiliza un sistema gestor de bases de datos (SGBD) para gestionar las operaciones cuando **múltiples usuarios o procesos intentan leer o modificar los mismos datos al mismo tiempo**.

Una **transacción** es una **secuencia de operaciones** de lectura y actualización de datos que forman una **unidad lógica** y que tienen que **ejecutarse como un todo**.
Las bases de datos relacionales proporcionan, en general, un completo y excelente soporte para transacciones, mientras que las bases de datos NoSQL no proporcionan, en general, soporte para transacciones.

Control de accesos concurrentes y transacciones

Las características que debe satisfacer una transacción se suelen resumir con el acrónimo ACID**, propiedades ACID**:

1. Atomicidad (*Atomicity*)

"O se hace todo, o no se hace nada."

2. Consistencia (*Consistency*)

"La base de datos siempre pasa de un estado válido a otro estado válido."

3. Isolation / Aislamiento (*Isolation*)

"Las transacciones concurrentes no interfieren entre sí.“

4. Durabilidad (*Durability*)

"Una vez guardado, el cambio es permanente."

Control de accesos concurrentes y transacciones

Los 3 problemas clásicos que evita:

- **Pérdida de actualizaciones (Lost Update):** Dos transacciones leen el mismo dato. La Transacción A modifica el valor y guarda; inmediatamente después, la Transacción B modifica su copia inicial y guarda, **sobrescribiendo y borrando** la modificación de A sin saberlo.

- **Lectura sucia (Dirty Read):** La Transacción A modifica un dato sin haber hecho COMMIT. La Transacción B lee ese dato modificado. Si A realiza un ROLLBACK (cancela la operación), los datos que leyó B nunca existieron realmente.

- **Lectura no repetible (Non-repeatable Read):** La Transacción A lee un registro. La Transacción B modifica o elimina ese registro y hace COMMIT. Si A vuelve a leer el mismo registro en la misma transacción, obtiene un valor diferente o descubre que el registro desapareció.

Control de accesos concurrentes y transacciones

Todas las ventajas que proporcionan las transacciones tienen contra partida un **menor rendimiento**, aunque los gestores de bases de datos hoy en día están muy optimizados y son muy escalables, lo que significa que el rendimiento se puede mejorar añadiendo los recursos de hardware y software necesarios y realizando cambios en la configuración del sistema.

En cualquier caso, las ventajas que ofrecen las transacciones pueden no ser estrictamente necesarias y no compensar la merma en el rendimiento para algunas aplicaciones.

Las **bases de datos NoSQL**, por ejemplo, renuncian a un soporte completo de transacciones a cambio de una **mayor disponibilidad y rendimiento**.

Persistencia de datos en ficheros

La **persistencia de datos en ficheros** es la capacidad de almacenar la información generada o manipulada por una aplicación en archivos físicos del sistema de archivos, de modo que los datos **permanezcan guardados tras el cierre de la aplicación o el apagado del sistema**.

A diferencia de las bases de datos relacionales o NoSQL, la persistencia en ficheros es la forma más directa, básica y de bajo nivel de guardar datos sin depender de un gestor de bases de datos (SGBD).

Este tipo de sistemas:

- tienen **limitaciones** para evitar la **redundancia e inconsistencia** de los datos, así como para definir y preservar restricciones de integridad.

- **no son apropiados** para dar servicio a **muchas aplicaciones concurrentes**, como no sea que las operaciones que realicen sean, en general, solo de lectura y muy ocasionalmente de modificación de datos.

Persistencia de datos en bases de datos relacionales

Los sistemas de bases de datos relacionales son los más utilizados en la actualidad para el almacenamiento de datos.

- Tienen sólidos fundamentos matemáticos, al estar basados en un elegante modelo formal: el modelo relacional.
- Son sencillos e intuitivos.
- La estructura básica de almacenamiento que proporcionan es la tabla. Todo se almacena en tablas.

Para su uso existe un lenguaje estándar y soportado universalmente: **SQL**

Mediante SQL se pueden crear esquemas relacionales (tablas, relaciones), definir restricciones de integridad para tablas individuales y para pares de tablas relacionadas (para evitar, por ejemplo, que se borre un cliente si existen facturas para ese cliente).

Los esquemas relacionales normalizados son simples y concisos, y permiten evitar redundancias e inconsistencias de datos, a la vez que se optimiza el espacio de almacenamiento utilizado.

Persistencia de datos en bases de datos relacionales

- Actualmente las actuales bases de datos relacionales son **muy escalables**, lo que significa que son apropiadas para gestionar bases de datos pequeñas con un pequeño volumen de operaciones, pero también para bases de datos enormes que soportan un continuo y elevado volumen de operaciones simultáneas de consulta y modificación de datos.
- Tienen un magnifico **soporte para transacciones**.
- Tienen mecanismos sofisticados de **copia de seguridad y de recuperación** que permiten recuperar los datos ante fallos de cualquier tipo.
- Existen **API** para bases de datos relacionales para todos los lenguajes de programación ampliamente utilizados. *Por ejemplo, JDBC para el lenguaje Java.*

Persistencia de datos en bases de datos NoSQL

El auge de las bases de datos NoSQL, en la actualidad, se debe principalmente a la **necesidad de recopilar, gestionar y analizar gigantescos conjuntos de datos heterogéneos**, que crecen continuamente a una velocidad cada vez mayor, esto es lo que se conoce como **big data**.

Este tipo de bases de datos da también respuesta a las necesidades de aplicaciones que ofrecen servicio, a través de la web, a un enorme y creciente número de usuarios que no solo consultan información, sino que la añaden y la modifican de manera continua.

NoSQL es todo aquello que **NO** se adhiere 100% al modelo relacional y a sus características:

1. Almacenamiento basado en tablas.
2. SQL como lenguaje para interactuar con la base de datos.
3. Soporte para restricciones de integridad y transacciones.

Persistencia de datos en bases de datos NoSQL

Algunas características de estas bases de datos son:

- El almacenamiento no está basado en tablas. En general, utilizan **estructuras de almacenamiento muy sencillas y muy flexibles**. Pueden ser arrays asociativos o de tipo clave-valor (como Redis), o documentos (como MongoDB), o basadas en columnas (como Casandra), o basadas en grafos, como otras bases de datos, por poner solo algunos ejemplos.

- **No se manejan con SQL** ni nada que se le parezca. Cada una tiene su propio lenguaje.

- En cuanto a **restricciones de integridad y transacciones** se puede decir que, en lugar de primar las características de las transacciones ACID (atomicidad, consistencia, aislamiento y durabilidad), prima la disponibilidad, lo que a veces se resume con el acrónimo BASE (basic anilability, soft state, eventual consistency), es decir: **disponibilidad básica, estado flexible y consistencia con el tiempo.**

Hemos acabado ….

¡¡ Vamos a repasar !!